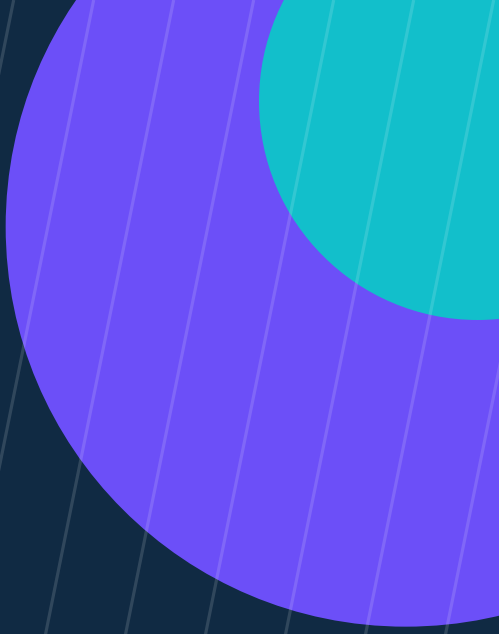




BEGINNER FRIENDLY

Git Commands Handbook

Section-wise explanations
with practical, real-world examples



```
$ git status
```

```
On branch main
```

```
nothing to commit, working tree clean
```



-by Atul Gupta

How to use this handbook

Learn the everyday workflow first: inspect, review, stage, commit, fetch, and push. Then move into branching and collaboration. History rewriting and low-level commands are clearly marked as advanced so you can return to them when the basics feel natural.

1. Working folder

The files you are currently editing on your computer.

2. Staging area

The exact changes you are preparing for the next snapshot.

3. Commit history

Saved snapshots that let you understand and recover the project's past.

A safe everyday workflow

STEP	ACTION	COMMAND
01	Inspect	<code>git status</code>
02	Review edits	<code>git diff</code>
03	Stage selected work	<code>git add <file></code>
04	Review staged work	<code>git diff --cached</code>
05	Save a snapshot	<code>git commit -m "Clear message"</code>
06	Download updates	<code>git fetch origin</code>
07	Publish your branch	<code>git push -u origin <branch></code>

Safety habit

Run `git status` before and after commands that change files or history. Review with `git diff` before committing.

Contents

01	Setup and configuration 11 commands
02	Creating and cloning repositories 6 commands
03	Checking repository status 5 commands
04	Staging files 5 commands
05	Committing changes 6 commands
06	Branch management 13 commands
07	Remote repositories 10 commands
08	Pushing and pulling 11 commands
09	Fetching changes 6 commands
10	Merging branches 6 commands
11	Rebasing commits 7 commands
12	History and logs 16 commands
13	Comparing changes 10 commands
14	Stashing work 8 commands
15	Undoing changes 7 commands
16	Cherry picking commits 5 commands

-
- | | |
|----|----------------------------------|
| 17 | Tags and releases
10 commands |
|----|----------------------------------|
-
- | | |
|----|--|
| 18 | Troubleshooting and recovery
7 commands |
|----|--|
-
- | | |
|----|-------------------------------|
| 19 | File management
7 commands |
|----|-------------------------------|
-
- | | |
|----|--------------------------|
| 20 | Submodules
7 commands |
|----|--------------------------|
-
- | | |
|----|--------------------------------------|
| 21 | Git Large File Storage
6 commands |
|----|--------------------------------------|
-
- | | |
|----|--|
| 22 | Advanced and low level commands
20 commands |
|----|--|
-

SECTION 01

Setup and configuration

Set up Git and inspect the configuration that controls its behavior.

git --version

Shows the installed Git version.

EXAMPLE

```
git --version
```

What happens: You may see git version 2.51.0. Use this first when checking whether Git is installed.

git config --list

Lists the configuration values Git can currently see.

EXAMPLE

```
git config --list
```

What happens: Review your name, email, editor, aliases, and other settings.

git config --global

Reads or writes a setting for your user account.

EXAMPLE

```
git config --global user.name "Atul Gupta"
```

What happens: Sets your name for commits in all repositories on this computer.

git config --local

Reads or writes a setting only for the current repository.

EXAMPLE

```
git config --local user.email "atul@company.com"
```

What happens: Overrides the global email in this repository. Run it inside the repository.

git config --system

Reads or writes a computer-wide setting for every user.

EXAMPLE

```
git config --system core.autocrlf true
```

What happens: Usually needs administrator permission. Beginners rarely need this scope.

git config --get

Prints one configuration value.

EXAMPLE

```
git config --get user.email
```

What happens: Confirms the email Git will put in new commits.

git config --unset

Removes a configuration value.

EXAMPLE

```
git config --global --unset user.email
```

What happens: Removes the global email. Check the scope before running it.

git config --edit

Opens a configuration file in your configured editor.

EXAMPLE

```
git config --global --edit
```

What happens: Lets you review and edit global settings directly.

git config --show-origin --list

Lists settings and the file from which each one came.

EXAMPLE

```
git config --show-origin --list
```

What happens: Useful when a local value unexpectedly overrides a global value.

git help

Opens detailed help for a Git command.

EXAMPLE

```
git help commit
```

What happens: Shows the manual for git commit. Press q to leave many terminal help viewers.

git help -a

Lists available Git commands.

EXAMPLE

```
git help -a
```

What happens: Use it when you remember what a command does but not its exact name.

SECTION 02

Creating and cloning repositories

Start a new repository or copy an existing one.

git init

Creates an empty Git repository in the current folder.

EXAMPLE

```
mkdir website && cd website  
git init
```

What happens: Git creates a hidden .git folder and starts tracking history for this project.

git clone

Downloads a repository and its history.

EXAMPLE

```
git clone https://github.com/octocat/Hello-World.git
```

What happens: Creates a Hello-World folder and configures origin automatically.

git clone --branch

Clones a repository and checks out a chosen branch or tag.

EXAMPLE

```
git clone --branch develop https://github.com/example/shop.git
```

What happens: The new working folder opens on develop.

git clone --depth

Downloads only a limited amount of recent history.

EXAMPLE

```
git clone --depth 1 https://github.com/example/shop.git
```

What happens: Creates a faster, smaller shallow clone with the newest commit only.

git clone --single-branch

Downloads history for one branch instead of all branches.

EXAMPLE

```
git clone --single-branch --branch main https://github.com/example/shop.git
```

What happens: Useful for CI jobs or when old branch history is unnecessary.

git clone --recurse-submodules

Clones the main repository and initializes its submodules.

EXAMPLE

```
git clone --recurse-submodules https://github.com/example/app.git
```

What happens: Also downloads repositories recorded as submodules.

SECTION 03

Checking repository status

See your current branch and the state of files before you commit.

git status

Shows changed, staged, untracked, and conflicted files.

EXAMPLE

```
git status
```

What happens: This is the safest command to run whenever you are unsure what Git sees.

git status --short

Shows a compact two-column status.

EXAMPLE

```
git status --short
```

What happens: M means modified, A means added, D means deleted, and ?? means untracked.

git status --branch

Adds branch and tracking information to status output.

EXAMPLE

```
git status --short --branch
```

What happens: Shows whether your branch is ahead of or behind its remote branch.

git branch --show-current

Prints only the current branch name.

EXAMPLE

```
git branch --show-current
```

What happens: Example output is main. This is the reliable replacement for the poster's nonstandard git summary entry.

git describe

Creates a readable name for the current commit from the nearest tag.

EXAMPLE

```
git describe --tags --always
```

What happens: You may see v1.4.0-3-g8a12c9d, meaning three commits after tag v1.4.0.

SECTION 04

Staging files

Choose exactly which changes will go into the next commit.

git add file

Stages one file.

EXAMPLE

```
git add src/login.js
```

What happens: Only changes in src/login.js are added to the staging area.

git add .

Stages changes under the current folder.

EXAMPLE

```
git add .
```

What happens: From the repository root, this usually stages all project changes. Check git status afterward.

git add --all

Stages new, modified, and deleted files across the repository.

EXAMPLE

```
git add --all
```

What happens: This is the clearest command when you intend to stage everything.

git add --update

Stages modifications and deletions of tracked files, but not new files.

EXAMPLE

```
git add --update
```

What happens: Useful when you want to exclude newly created files.

git add --patch

Lets you stage selected pieces of a changed file.

EXAMPLE

```
git add --patch src/login.js
```

What happens: Answer y for a hunk you want in the commit and n for one you want to leave unstaged.

SECTION 05

Committing changes

Save a meaningful snapshot after reviewing the staged changes.

`git commit`

Creates a commit from staged changes and opens an editor for the message.

EXAMPLE

```
git commit
```

What happens: Write a short first line explaining what changed, save, and close the editor.

`git commit -m`

Creates a commit with a message written in the command.

EXAMPLE

```
git commit -m "Add login validation"
```

What happens: Creates a commit without opening an editor.

`git commit -a`

Stages modifications and deletions of tracked files, then commits them.

EXAMPLE

```
git commit -a -m "Fix checkout totals"
```

What happens: It does not include untracked files; use `git add` for new files.

`git commit --amend`

Replaces the most recent local commit.

EXAMPLE

```
git add README.md  
git commit --amend -m "Improve setup instructions"
```

What happens: Adds the staged fix and changes the last commit. Avoid amending a commit others already use.

git commit --amend --no-edit

Adds staged changes to the last commit without changing its message.

EXAMPLE

```
git add forgotten-file.txt
git commit --amend --no-edit
```

What happens: Useful immediately after noticing a forgotten file.

git commit --allow-empty

Creates a commit even when no files changed.

EXAMPLE

```
git commit --allow-empty -m "Trigger deployment"
```

What happens: Often used to trigger automation; it is uncommon in normal coding work.

SECTION 06

Branch management

Create separate lines of work and move between them safely.

git branch

Lists local branches.

EXAMPLE

```
git branch
```

What happens: The branch marked with an asterisk is the current branch.

git branch --all

Lists local and remote-tracking branches.

EXAMPLE

```
git branch --all
```

What happens: Remote-tracking branches normally appear under remotes/origin.

git branch --remotes

Lists only remote-tracking branches.

EXAMPLE

```
git branch --remotes
```

What happens: Useful for seeing branches published on the remote.

git branch -d

Deletes a local branch only if Git considers it merged.

EXAMPLE

```
git branch -d feature/login
```

What happens: Safe cleanup after the feature has been merged.

git branch -D

Forcibly deletes a local branch.

EXAMPLE

```
git branch -D experiment
```

What happens: Unmerged commits may become difficult to find. Use only when you are sure.

git branch -m

Renames a branch.

EXAMPLE

```
git branch -m master main
```

What happens: Renames the current branch from master to main.

git switch

Moves to an existing branch.

EXAMPLE

```
git switch develop
```

What happens: Updates the working folder to match develop.

git switch -

Returns to the previously checked-out branch.

EXAMPLE

```
git switch -
```

What happens: Quickly toggles between two branches.

git switch --create

Creates a new branch and moves to it.

EXAMPLE

```
git switch --create feature/payment
```

What happens: Modern, clear alternative to git checkout -b.

git checkout

Older command that can switch branches or restore paths.

EXAMPLE

```
git checkout develop
```

What happens: Still common in tutorials, but git switch is clearer for branch changes.

git checkout -b

Creates a branch and moves to it.

EXAMPLE

```
git checkout -b feature/payment
```

What happens: Older equivalent of git switch -c feature/payment.

git checkout -- file

Restores a file from the index using older syntax.

EXAMPLE

```
git checkout -- README.md
```

What happens: Discards unstaged edits in README.md. Prefer git restore README.md for clarity.

git checkout --force

Forces a checkout and discards obstructing local changes.

EXAMPLE

```
git checkout --force main
```

What happens: Destructive. Commit or stash valuable work first.

SECTION 07

Remote repositories

Manage the named connections between your local repository and servers such as GitHub.

git remote

Lists remote names.

EXAMPLE

```
git remote
```

What happens: Most cloned repositories show origin.

git remote -v

Lists remotes with fetch and push URLs.

EXAMPLE

```
git remote -v
```

What happens: Confirms which server receives your pushes.

git remote add

Adds a new remote connection.

EXAMPLE

```
git remote add origin https://github.com/atul/shop.git
```

What happens: Names the URL origin so later commands can use the short name.

git remote remove

Removes a remote name and its tracking references.

EXAMPLE

```
git remote remove old-server
```

What happens: Does not delete the repository on the server.

git remote rename

Renames a remote.

EXAMPLE

```
git remote rename origin upstream
```

What happens: All related remote-tracking names are updated.

git remote set-url

Changes a remote's URL.

EXAMPLE

```
git remote set-url origin git@github.com:atul/shop.git
```

What happens: Often used when switching from HTTPS to SSH.

git remote show

Shows detailed information about one remote.

EXAMPLE

```
git remote show origin
```

What happens: Displays tracked branches, push targets, and stale references.

git remote get-url

Prints a remote's URL.

EXAMPLE

```
git remote get-url origin
```

What happens: A quick way to confirm the exact repository address.

git remote prune

Removes stale remote-tracking branches.

EXAMPLE

```
git remote prune origin
```

What happens: Cleans local references to branches deleted from origin.

git remote update

Fetches updates for configured remotes.

EXAMPLE

```
git remote update
```

What happens: Updates remote-tracking references without merging them into your branch.

SECTION 08

Pushing and pulling

Exchange commits with a remote repository.

git push

Uploads commits from the current branch to its configured upstream.

EXAMPLE

```
git push
```

What happens: Works after an upstream relationship has been set.

git push origin main

Uploads local main to main on origin.

EXAMPLE

```
git push origin main
```

What happens: A clear first push when no upstream is configured.

git push --set-upstream

Pushes a branch and remembers its remote branch.

EXAMPLE

```
git push --set-upstream origin feature/login
```

What happens: Later, plain git push and git pull work on that branch.

git push --all

Pushes all local branches to a remote.

EXAMPLE

```
git push origin --all
```

What happens: Use carefully; it may publish branches you meant to keep local.

git push --tags

Pushes all local tags.

EXAMPLE

```
git push origin --tags
```

What happens: Publishes release tags that a normal branch push does not send.

git push --force

Replaces remote history with local history.

EXAMPLE

```
git push --force origin feature/rewrite
```

What happens: Risky on shared branches because it can delete other people's remote commits.

git push --force-with-lease

Force-pushes only if the remote still matches your last known state.

EXAMPLE

```
git push --force-with-lease origin feature/rewrite
```

What happens: Safer than --force after a rebase because it refuses to overwrite unseen work.

git push --delete

Deletes a branch from a remote.

EXAMPLE

```
git push origin --delete feature/old-ui
```

What happens: The local branch remains until you delete it separately.

git pull

Fetches remote commits and integrates them using configured behavior.

EXAMPLE

```
git pull
```

What happens: It may merge or rebase. Beginners should know which behavior their repository uses.

git pull --rebase

Fetches, then replays local commits on top of the remote branch.

EXAMPLE

```
git pull --rebase origin main
```

What happens: Keeps a linear history but may require conflict resolution.

git pull --no-rebase

Fetches, then merges the remote branch.

EXAMPLE

```
git pull --no-rebase origin main
```

What happens: May create a merge commit when local and remote history diverge.

SECTION 09

Fetching changes

Download remote information without immediately changing your working branch.

git fetch

Downloads new commits and references from the default remote.

EXAMPLE

```
git fetch
```

What happens: Your current branch and files do not change.

git fetch origin

Fetches from a specific remote.

EXAMPLE

```
git fetch origin
```

What happens: Updates origin/main and other configured origin references.

git fetch --all

Fetches from every configured remote.

EXAMPLE

```
git fetch --all
```

What happens: Useful when a project has origin and upstream remotes.

git fetch --prune

Fetches updates and removes stale remote-tracking references.

EXAMPLE

```
git fetch --prune origin
```

What happens: Keeps the remote branch list tidy.

git fetch --tags

Fetches tags in addition to normal branch updates.

EXAMPLE

```
git fetch --tags origin
```

What happens: Useful before checking out a newly published release tag.

git fetch --depth

Fetches only a limited history depth in a shallow repository.

EXAMPLE

```
git fetch --depth 50 origin main
```

What happens: Deepens the available history to 50 commits.

SECTION 10

Merging branches

Combine completed work from one branch into another.

git merge

Merges a named branch into the current branch.

EXAMPLE

```
git switch main  
git merge feature/login
```

What happens: main receives the commits from feature/login.

git merge --no-ff

Creates a merge commit even when a fast-forward is possible.

EXAMPLE

```
git merge --no-ff feature/login
```

What happens: Preserves a visible boundary around the feature's history.

git merge --ff-only

Merges only if the current branch can move forward without a merge commit.

EXAMPLE

```
git merge --ff-only origin/main
```

What happens: Fails safely when histories have diverged.

git merge --squash

Places the other branch's combined changes in the staging area without its individual commits.

EXAMPLE

```
git merge --squash feature/prototype  
git commit -m "Add prototype"
```

What happens: Creates one new commit containing the feature's net changes.

git merge --abort

Cancels an in-progress conflicted merge.

EXAMPLE

```
git merge --abort
```

What happens: Returns as closely as possible to the state before the merge began.

git merge --continue

Finishes a merge after conflicts are resolved and staged.

EXAMPLE

```
git add src/login.js  
git merge --continue
```

What happens: Git creates or completes the merge commit.

SECTION 11

Rebasing commits

Move or clean up commits by replaying them on a new base.

Pause and check

Some commands in this section can rewrite history or discard work. Read the result note, check git status, and make a backup branch when unsure.

git rebase

Replays the current branch's commits on another branch.

EXAMPLE

```
git switch feature/login  
git rebase main
```

What happens: The feature appears to start from the newest main. Commit IDs change.

git rebase --interactive

Opens a list of commits so you can reorder, edit, squash, or reword them.

EXAMPLE

```
git rebase -i HEAD~3
```

What happens: Lets you clean the last three local commits before sharing them.

git rebase --continue

Continues after you resolve and stage rebase conflicts.

EXAMPLE

```
git add src/login.js  
git rebase --continue
```

What happens: Git proceeds to the next commit being replayed.

git rebase --abort

Cancels the entire in-progress rebase.

EXAMPLE

```
git rebase --abort
```

What happens: Returns the branch to its state before the rebase started.

git rebase --skip

Drops the current problematic commit and continues.

EXAMPLE

```
git rebase --skip
```

What happens: Use only when that commit's change is unnecessary or already present.

git rebase --onto

Replays a selected range of commits onto a different base.

EXAMPLE

```
git rebase --onto main server client
```

What happens: Moves commits after server through client onto main. This is advanced; inspect the graph first.

git rebase --exec

Runs a command after each replayed commit.

EXAMPLE

```
git rebase -i --exec "npm test" HEAD~4
```

What happens: Stops when a test fails so you can fix that point in history.

SECTION 12

History and logs

Inspect commits, authors, messages, patches, and recovery history.

git log

Shows commit history for the current branch.

EXAMPLE

```
git log
```

What happens: Displays commit IDs, authors, dates, and messages.

git log --oneline

Shows one compact line per commit.

EXAMPLE

```
git log --oneline -10
```

What happens: Lists the newest ten commits using abbreviated IDs.

git log --graph

Draws an ASCII branch and merge graph.

EXAMPLE

```
git log --graph --oneline --decorate
```

What happens: Makes branch structure easier to understand.

git log --all

Shows history reachable from all references.

EXAMPLE

```
git log --all --oneline --graph
```

What happens: Includes commits from branches other than the current one.

git log --stat

Shows a file-change summary for each commit.

EXAMPLE

```
git log --stat -3
```

What happens: Shows which files changed and how many lines were added or removed.

git log --patch

Shows the complete patch for each selected commit.

EXAMPLE

```
git log -p -1
```

What happens: Displays the exact line changes in the newest commit.

git log --author

Filters commits by author name or email pattern.

EXAMPLE

```
git log --author="Atul" --oneline
```

What happens: Shows commits whose author matches Atul.

git log --grep

Filters commits by message text.

EXAMPLE

```
git log --grep="login" --oneline
```

What happens: Finds commits with login in their commit message.

git log --since

Shows commits after a date or relative time.

EXAMPLE

```
git log --since="2 weeks ago" --oneline
```

What happens: Limits the log to recent work.

git log --until

Shows commits before a date.

EXAMPLE

```
git log --until="2026-10-01" --oneline
```

What happens: Useful with --since to define a date range.

git log --follow

Follows one file across renames when possible.

EXAMPLE

```
git log --follow -- src/app.js
```

What happens: Shows the file's history even if it had an earlier name.

git shortlog

Summarizes commits by author.

EXAMPLE

```
git shortlog -sn --all
```

What happens: Counts commits per author and sorts by count.

git reflog

Shows recent movements of local references such as HEAD.

EXAMPLE

```
git reflog
```

What happens: Often helps recover a commit after reset, rebase, or an accidental branch deletion.

git blame

Shows the last commit and author for each line of a file.

EXAMPLE

```
git blame -L 20,35 src/app.js
```

What happens: Examines who last changed lines 20 through 35 and in which commit.

git bisect

Uses binary search to find the commit that introduced a bug.

EXAMPLE

```
git bisect start
git bisect bad
git bisect good v1.0.0
```

What happens: Test each chosen commit and mark it good or bad until Git identifies the first bad commit.

git show

Displays a Git object, usually a commit and its patch.

EXAMPLE

```
git show HEAD
```

What happens: Shows the newest commit's metadata and exact changes.

SECTION 13

Comparing changes

Compare the working folder, staging area, commits, branches, and files.

git diff

Shows unstaged changes in tracked files.

EXAMPLE

```
git diff
```

What happens: Compares the working folder with the staging area.

git diff --cached

Shows staged changes that will enter the next commit.

EXAMPLE

```
git diff --cached
```

What happens: Review this before committing.

git diff --stat

Shows a compact file and line-count summary.

EXAMPLE

```
git diff --stat
```

What happens: Useful when the full patch is too long.

git diff --name-only

Lists only changed file names.

EXAMPLE

```
git diff --name-only main...feature/login
```

What happens: Quickly shows which files the feature changed.

git diff --name-status

Lists changed files with status letters.

EXAMPLE

```
git diff --name-status HEAD~1 HEAD
```

What happens: A means added, M modified, D deleted, and R renamed.

git diff --check

Checks changes for whitespace errors.

EXAMPLE

```
git diff --check
```

What happens: Reports problems such as trailing spaces before you commit.

git diff HEAD

Shows all tracked changes since the current commit.

EXAMPLE

```
git diff HEAD
```

What happens: Includes both staged and unstaged changes.

git diff branch1..branch2

Compares the tips of two branches.

EXAMPLE

```
git diff main..feature/login
```

What happens: Shows how feature/login's current files differ from main's current files.

git diff commit1 commit2

Compares two exact commits.

EXAMPLE

```
git diff a1b2c3d f6e7d8c
```

What happens: Use commit IDs copied from git log --oneline.

git diff -- file

Limits a comparison to one path.

EXAMPLE

```
git diff main -- src/login.js
```

What happens: Shows only changes to src/login.js compared with main.

SECTION 14

Stashing work

Temporarily set aside unfinished changes so you can switch tasks.

git stash

Saves tracked changes and restores a clean working folder.

EXAMPLE

```
git stash
```

What happens: Equivalent to a basic git stash push.

git stash push

Creates a stash, optionally with a message or path selection.

EXAMPLE

```
git stash push -m "login form draft"
```

What happens: A clear message makes the stash easy to recognize later.

git stash list

Lists saved stashes.

EXAMPLE

```
git stash list
```

What happens: The newest is normally stash@{0}.

git stash show

Shows a summary or patch for a stash.

EXAMPLE

```
git stash show -p stash@{0}
```

What happens: Displays the exact changes in the newest stash.

git stash apply

Applies a stash but keeps it in the stash list.

EXAMPLE

```
git stash apply stash@{1}
```

What happens: Good when you want a backup until the restored work is verified.

git stash pop

Applies a stash and removes it if the operation succeeds cleanly.

EXAMPLE

```
git stash pop
```

What happens: May produce conflicts; resolve them like merge conflicts.

git stash drop

Deletes one stash.

EXAMPLE

```
git stash drop stash@{2}
```

What happens: Confirm the stash number first because normal recovery is difficult.

git stash branch

Creates a branch from the commit where a stash was made and applies it.

EXAMPLE

```
git stash branch fix/login stash@{0}
```

What happens: Helpful when the original branch changed and a normal apply conflicts.

SECTION 15

Undoing changes

Use the least destructive command that solves the problem.

Pause and check

Some commands in this section can rewrite history or discard work. Read the result note, check git status, and make a backup branch when unsure.

git restore

Discards unstaged changes in a file.

EXAMPLE

```
git restore src/app.js
```

What happens: The file returns to the staged or committed version. Uncommitted edits are lost.

git restore --staged

Removes a file from the staging area while keeping its edits.

EXAMPLE

```
git restore --staged secrets.txt
```

What happens: Use this when you staged the wrong file.

git reset

Moves the current branch and can also change the index and working folder.

EXAMPLE

```
git reset HEAD~1
```

What happens: Default mixed mode removes the last commit but keeps its changes unstaged.

git reset --soft

Moves the branch while keeping changes staged.

EXAMPLE

```
git reset --soft HEAD~1
```

What happens: Useful for replacing the last local commit with a better one.

git reset --mixed

Moves the branch and keeps changes in the working folder but unstaged.

EXAMPLE

```
git reset --mixed HEAD~1
```

What happens: This is the default reset mode.

git reset --hard

Moves the branch and discards tracked changes in the index and working folder.

EXAMPLE

```
git reset --hard HEAD~1
```

What happens: Destructive. Create a backup branch or stash first if anything matters.

git revert

Creates a new commit that reverses an earlier commit.

EXAMPLE

```
git revert a1b2c3d
```

What happens: Best for shared history because it does not rewrite existing commits.

SECTION 16

Cherry picking commits

Apply selected commits to the current branch.

git cherry-pick

Copies the change from a named commit onto the current branch.

EXAMPLE

```
git switch release  
git cherry-pick a1b2c3d
```

What happens: Creates a new commit on release with the same change.

git cherry-pick --continue

Continues after cherry-pick conflicts are resolved and staged.

EXAMPLE

```
git add src/app.js  
git cherry-pick --continue
```

What happens: Finishes applying the selected commit.

git cherry-pick --abort

Cancels an in-progress cherry-pick.

EXAMPLE

```
git cherry-pick --abort
```

What happens: Returns the branch to its earlier state.

git cherry-pick --skip

Skips the current commit in a multi-commit cherry-pick.

EXAMPLE

```
git cherry-pick --skip
```

What happens: Use only when the skipped change is unnecessary or already present.

git cherry-pick --no-commit

Applies the change without creating a commit.

EXAMPLE

```
git cherry-pick --no-commit a1b2c3d
```

What happens: Lets you edit or combine the result before committing.

SECTION 17

Tags and releases

Name important commits, usually to mark versions or releases.

git tag

Lists tags.

EXAMPLE

```
git tag
```

What happens: Shows version names such as v1.0.0 and v1.1.0.

git tag name

Creates a lightweight tag at the current commit.

EXAMPLE

```
git tag v1.0.0
```

What happens: The name points to HEAD without a message.

git tag --annotate

Creates an annotated tag with tagger information and a message.

EXAMPLE

```
git tag -a v1.0.0 -m "First stable release"
```

What happens: Annotated tags are usually preferred for releases.

git tag --delete

Deletes a local tag.

EXAMPLE

```
git tag -d v1.0.0-rc1
```

What happens: Does not delete the tag from a remote.

git tag --list

Lists tags matching a pattern.

EXAMPLE

```
git tag -l "v1.*"
```

What happens: Shows tags whose names start with v1.

git tag --contains

Lists tags that contain a commit.

EXAMPLE

```
git tag --contains a1b2c3d
```

What happens: Helps identify releases that include a fix.

git tag --merged

Lists tags reachable from a branch or commit.

EXAMPLE

```
git tag --merged main
```

What happens: Shows tags whose commits are already in main.

git push origin tag

Pushes one tag.

EXAMPLE

```
git push origin v1.0.0
```

What happens: Publishes the selected release tag.

git push --tags

Pushes all local tags missing from the remote.

EXAMPLE

```
git push origin --tags
```

What happens: Review local tags first to avoid publishing unwanted ones.

git push --delete tag

Deletes a tag from a remote.

EXAMPLE

```
git push origin --delete v1.0.0-rc1
```

What happens: Delete the local copy separately with `git tag -d` if required.

SECTION 18

Troubleshooting and recovery

Check repository integrity, clean generated files, find lost work, and verify signed objects.

git blame

Finds the last commit that changed each line.

EXAMPLE

```
git blame src/payment.js
```

What happens: Use the commit ID to read the reason in git show; do not treat blame as proof of fault.

git bisect

Finds the first bad commit through repeated good or bad tests.

EXAMPLE

```
git bisect start
git bisect bad HEAD
git bisect good v2.0.0
```

What happens: When finished, run git bisect reset.

git fsck

Checks object connectivity and validity.

EXAMPLE

```
git fsck --full
```

What happens: Useful when repository corruption is suspected or when searching for dangling objects.

git reflog

Shows where local references pointed recently.

EXAMPLE

```
git reflog --date=local
```

What happens: Create a rescue branch with git branch rescue <commit-id>.

git show

Inspects a commit, tag, tree, or blob.

EXAMPLE

```
git show a1b2c3d:README.md
```

What happens: Prints README.md exactly as it existed in that commit.

git verify-commit

Checks the cryptographic signature on a signed commit.

EXAMPLE

```
git verify-commit a1b2c3d
```

What happens: Requires the signer's public key to be available.

git verify-tag

Checks the cryptographic signature on a signed annotated tag.

EXAMPLE

```
git verify-tag v1.0.0
```

What happens: Use it before trusting a signed release tag.

SECTION 19

File management

Rename, remove, list, and diagnose tracked paths.

git rm

Removes a tracked file and stages the deletion.

EXAMPLE

```
git rm old-config.yml
```

What happens: The next commit records the file's removal.

git rm --cached

Stops tracking a file but keeps it in the working folder.

EXAMPLE

```
git rm --cached .env
```

What happens: Add .env to .gitignore before committing so it stays untracked.

git mv

Moves or renames a tracked file and stages the change.

EXAMPLE

```
git mv notes.txt docs/notes.txt
```

What happens: Git records the delete and add and may display them as a rename.

git ls-files

Lists files in the index.

EXAMPLE

```
git ls-files
```

What happens: Shows files Git currently tracks or has staged.

git ls-tree

Lists files and folders stored in a tree object.

EXAMPLE

```
git ls-tree -r --name-only HEAD
```

What happens: Lists every path in the current commit.

git check-ignore

Explains whether and why a path is ignored.

EXAMPLE

```
git check-ignore -v logs/app.log
```

What happens: Shows the matching .gitignore file and rule.

git check-attr

Shows Git attribute values for paths.

EXAMPLE

```
git check-attr -a -- README.md
```

What happens: Displays attributes applied through .gitattributes.

SECTION 20

Submodules

Track another Git repository at a specific commit inside the main repository.

`git submodule`

Shows registered submodules and their checked-out commits.

EXAMPLE

```
git submodule
```

What happens: A leading minus means the submodule is not initialized.

`git submodule add`

Adds another repository as a submodule.

EXAMPLE

```
git submodule add https://github.com/example/ui-kit.git vendor/ui-kit
```

What happens: Creates the submodule folder and updates `.gitmodules`.

`git submodule init`

Registers submodules listed in `.gitmodules` in local configuration.

EXAMPLE

```
git submodule init
```

What happens: This prepares them but does not download their content.

`git submodule update`

Checks out each submodule at the commit recorded by the main repository.

EXAMPLE

```
git submodule update --init --recursive
```

What happens: The combined form is the safest setup after cloning an existing project.

git submodule status

Shows each submodule's recorded and current commit.

EXAMPLE

```
git submodule status
```

What happens: Use it to see whether a submodule is initialized or modified.

git submodule foreach

Runs a shell command inside each submodule.

EXAMPLE

```
git submodule foreach git status --short
```

What happens: Checks for uncommitted changes in every submodule.

git submodule deinit

Unregisters a submodule from the working tree.

EXAMPLE

```
git submodule deinit -f vendor/ui-kit
```

What happens: The submodule definition remains in history; removing one fully needs additional steps.

SECTION 21

Git Large File Storage

Store large file content outside normal Git objects while keeping small pointer files in the repository

git lfs install

Sets up Git LFS filters for your user account.

EXAMPLE

```
git lfs install
```

What happens: Run once on a computer after installing Git LFS.

git lfs track

Adds a file pattern to .gitattributes for LFS tracking.

EXAMPLE

```
git lfs track "*.psd"
```

What happens: Commit the updated .gitattributes file with the project.

git lfs untrack

Stops assigning a pattern to LFS for future additions.

EXAMPLE

```
git lfs untrack "*.psd"
```

What happens: Existing LFS history is not automatically rewritten.

git lfs ls-files

Lists LFS-managed files in the current checkout.

EXAMPLE

```
git lfs ls-files
```

What happens: Confirms that intended large files are using LFS.

git lfs pull

Downloads LFS objects for the current checkout.

EXAMPLE

```
git lfs pull
```

What happens: Replaces pointer content with the required large files.

git lfs push

Uploads LFS objects to a remote.

EXAMPLE

```
git lfs push origin main
```

What happens: A normal git push usually handles this automatically; use it when objects are missing.

SECTION 22

Advanced and low level commands

Inspect Git's internal objects and references. These commands are valuable for learning and tooling

Pause and check

Some commands in this section can rewrite history or discard work. Read the result note, check git status, and make a backup branch when unsure.

git cat-file

Shows an object's type, size, or contents.

EXAMPLE

```
git cat-file -p HEAD
```

What happens: Pretty-prints the commit object referenced by HEAD.

git hash-object

Computes an object ID and can write a blob into the object database.

EXAMPLE

```
git hash-object README.md
```

What happens: Prints the hash Git would use for the file content.

git count-objects

Reports loose-object counts and disk usage.

EXAMPLE

```
git count-objects -vH
```

What happens: Shows human-readable repository storage information.

git for-each-ref

Formats information about references.

EXAMPLE

```
git for-each-ref --format='%(%(refname:short) %(objectname:short))' refs/heads
```

What happens: Lists local branches and their abbreviated commit IDs.

git rev-parse

Resolves revision names and parses revision-related arguments.

EXAMPLE

```
git rev-parse --show-toplevel
```

What happens: Prints the repository root folder.

git rev-list

Lists commit objects reachable from revisions.

EXAMPLE

```
git rev-list --count HEAD
```

What happens: Counts commits reachable from HEAD.

git read-tree

Reads tree information into the index.

EXAMPLE

```
git read-tree HEAD
```

What happens: Replaces index entries with the tree from HEAD. Advanced and potentially disruptive.

git write-tree

Creates a tree object from the current index.

EXAMPLE

```
git write-tree
```

What happens: Prints the new tree object's ID.

git update-index

Directly changes index information.

EXAMPLE

```
git update-index --assume-unchanged local.conf
```

What happens: Asks Git to avoid normal change checks for the path; it is not a security or ignore feature.

git update-ref

Safely creates, updates, or deletes a reference.

EXAMPLE

```
git update-ref refs/heads/backup HEAD
```

What happens: Creates or moves backup to the current commit.

git symbolic-ref

Reads or changes a symbolic reference.

EXAMPLE

```
git symbolic-ref --short HEAD
```

What happens: Prints the current branch name when HEAD is attached.

git diff-tree

Compares the content represented by tree objects.

EXAMPLE

```
git diff-tree --no-commit-id --name-only -r HEAD
```

What happens: Lists paths changed by the current commit.

git diff-files

Compares the working folder with the index.

EXAMPLE

```
git diff-files --name-status
```

What happens: Shows unstaged tracked-file changes using plumbing output.

git diff-index

Compares a tree with the working folder or index.

EXAMPLE

```
git diff-index --cached HEAD
```

What happens: Shows staged differences relative to HEAD.

git commit-tree

Creates a commit object from a tree and parent information.

EXAMPLE

```
echo 'Manual commit' | git commit-tree <tree-id> -p HEAD
```

What happens: Prints a commit ID but does not move a branch automatically.

git replace

Makes one object stand in for another locally.

EXAMPLE

```
git replace <old-commit> <new-commit>
```

What happens: Useful for temporary history repair or experimentation; replacements are local by default.

git mktag

Creates a tag object from validated tag text.

EXAMPLE

```
git mktag < tag-description.txt
```

What happens: Low-level alternative to git tag; the input must follow Git's tag-object format.

git pack-objects

Creates a packed archive of Git objects.

EXAMPLE

```
git rev-list --objects --all | git pack-objects pack
```

What happens: Creates pack files for low-level storage or transfer workflows.

git index-pack

Builds an index for a pack file and verifies its objects.

EXAMPLE

```
git index-pack objects.pack
```

What happens: Produces an index that lets Git locate objects inside the pack.

git unpack-objects

Expands objects from a pack into loose objects.

EXAMPLE

```
git unpack-objects < objects.pack
```

What happens: Usually used in repair or internal workflows, not routine development.

Quick reference

NEED	COMMAND
See what is happening	<code>git status</code>
Review unstaged edits	<code>git diff</code>
Stage one file	<code>git add <file></code>
Review staged edits	<code>git diff --cached</code>
Commit staged work	<code>git commit -m "message"</code>
Create and enter a branch	<code>git switch -c <branch></code>
Return to main	<code>git switch main</code>
Download remote updates	<code>git fetch origin</code>
Update with a rebase	<code>git pull --rebase</code>
Push a new branch	<code>git push -u origin <branch></code>
Temporarily save work	<code>git stash push -m "message"</code>
Restore stashed work	<code>git stash apply</code>
Unstage a file	<code>git restore --staged <file></code>
Discard unstaged edits	<code>git restore <file></code>
Undo a shared commit safely	<code>git revert <commit></code>
View a compact graph	<code>git log --graph --oneline --decorate --all</code>

Final safety checklist

CHECK Run `git status` before and after commands that change history or files.

CHECK Use `git diff` and `git diff --cached` before committing.

CHECK Prefer `git revert` for commits that have already been shared.

CHECK Prefer `--force-with-lease` to `--force` when a force push is truly necessary.

CHECK Treat `reset --hard`, `clean`, `checkout --force`, `branch -D`, and `stash drop` as destructive operations.